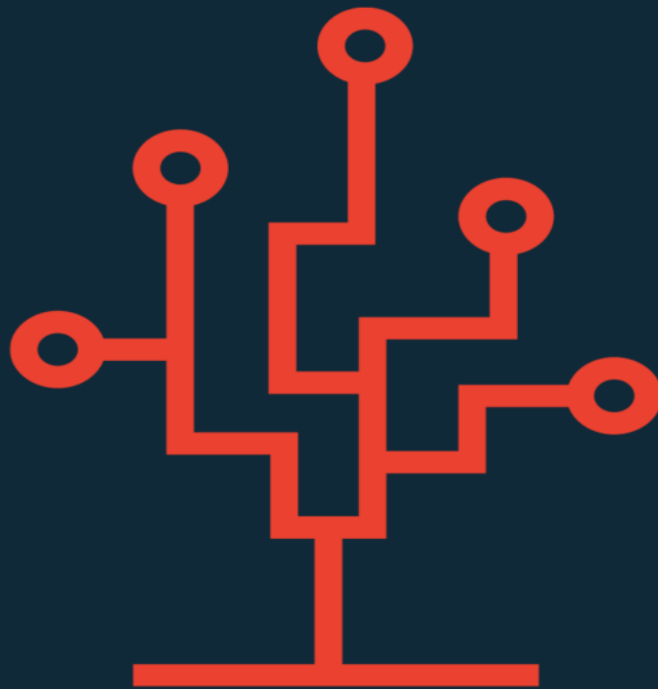


MICROSERVICES WITH LARAVEL



MARTIN JOO

This is a sample chapter from the original book.
Get your copy here: <https://microservices-laravel.io>

Introduction	4
What's the problem with monoliths?	4
Loose coupling	5
High cohesion	5
So, what are microservices?	6
What's exactly a service?	8
What makes it micro?	8
Finding bounded context	9
Storing data in different services	10
Communication between services	13
Sync communication	13
Event-driven communication	15
Frontend	16
The promise of microservices	18
Building a microservices application	20
Requirements	20
Create products	20
Fill up inventory	21
Show the product catalog	21
Search in the catalog	21
Show the product's details	22
Rating a product	22
Buying a product	22
High-level overview	23
Installing the project	26
Products service	30
Redis event streams	33
Add a new entry	34
Read from the stream	34
Publishing events	36
Consuming events	37
Back to the Product service	41
Ratings Service	47
Warehouse service	56
Catalog service	67
Aggregate Catalog service	68
Proxy Catalog service	72

No Catalog service at all	75
How to choose?	75
Implementation	77
Warehouse	79
Warehouse service API	81
Ratings	85
Ratings service API	87
Products	90
Products service API	94
Put everything together	98
Orders service	104
Resources vs Container	110
Docker Containers	111
Dockerfile	112
Docker-compose	116
API gateway	122
Nginx reverse proxy	122
Custom gateway	123
Implementation	125
Logging	129
Frontend	132
Bonus content	138
Testing	138
Setup	138
API tests	140
CI Pipeline	146
Final thoughts	152

Some theory

What's the problem with monoliths?

In order to understand what microservices are, first, we have to talk about monolith systems and their drawbacks.

Imagine Amazon as a Laravel project. It is one huge git repository, and it's made of classes like these:

- Product
- Inventory
- Catalog
- Rating
- Shipping
- Order
- OrderHistory
- Payment
- Discount
- Refund
- Recommendation
- Wishlist
- Account
- PrimeMember
- OneClickBuy

And so on. Just ask yourself: what does the one-click buy functionality have to do with shipping? One-click buy cares about credit card numbers, and user info, while shipping cares about product attributes, like weight and height, and addresses. Similarly refund logic has nothing to do with a wishlist.

Despite the fact that these modules, these classes are very different from each other, they live in the same repository, they can be accessed by the same API, they can call each other, and they can share some general functions or classes.

They are **tightly coupled** and have **low cohesion**. But in great architecture, we'd like to have **loose coupling** and **high cohesion**.

What the hell do those words mean?

Loose coupling

If a project is loosely coupled, a change in one place should not require a change in another place. Imagine you have Product, Order, Discount, and Payment in the same project. When you start the project you have some small, nice classes. The user creates an order with some product ids, you make a query from Product, calculate sum prices, and so on, nice and simple. After a while business wants discounts. So you call the Discount class from Order, and you pass the products. Nice and simple. One day, the business wants a discount based on the total amount of orders. No problem, you just pass the Order object to Discount. Next, you need to give a discount based on the user's order history and payments, so you pass more orders, and the User object to the Discount, maybe you also want to get something from the Payment module.

And here we are. The tightly coupled monolith. Now you have communication between:

- Order
- Discount
- Product
- Payment
- OrderHistory
- User

What happens next? You modify the *getNetPrice()* function in the Product class, and suddenly the Discount API call returns a 500 status code. Or even worse it calculates the wrong amounts. You change something in the User class and Order starts to behave differently. You modify something in OrderHistory and you get the wrong discounts.

This is a tightly coupled system. You make a change somewhere, and the application breaks in a different place. Oftentimes in a completely unrelated place.

High cohesion

High cohesion means that related behavior sits in one place, and other unrelated behavior sits in a different place. You can interpret this at the level of classes, for example, in the Order class, you don't want a method called *getProductPrice()*, you want it in the Product class. Soon, when we actually get to the point of microservices we will talk about cohesion at the level of product or services. But for now, the important part is we want related behavior in one place. And this is because, if we want to change that behavior we want to make sure to not break

anything else. So if you have high cohesion, you can make easier, more secure changes in your system.

In the world of monoliths, it is very easy to make high coupling and low cohesion. It comes from the nature of monoliths, you just put a ton of unrelated classes and methods next to each other. And as the number of features, the number of classes, methods, and the number of developers grow, it's just too easy to make the mistakes I mentioned above.

Don't get me wrong, you can write good, high-quality monoliths. And in my opinion, every system has some coupling and cohesion problems, no matter how well made it is.

What are microservices?

Imagine a restaurant where the waiter is using a mobile app to take your order. He takes your order, and the chef gets the information about what he has to cook. When the chef is done, the waiter delivers your meal, then you eat it. After that, the waiter prints your receipt via the mobile app. At the end of the day, the manager opens the same system on his laptop and sees the revenue, the net income, the cost of revenue and calls it a day. Every Friday he goes to his favorite supermarket and buys some ingredients, then he tracks them in the system. At the end of every month, he downloads an Excel about financials and e-mails it to the bookkeeper.

In this domain, we have models like:

- Product
- ProductGroup
- Ingredient
- Order
- OrderItem
- Receipt
- Discount
- Finance
- Storage
- Purchase

First, try to create some groups from these classes. In each group let's put similar classes, which are related to each other:

Groups:

- Product
- Order
- Discount
- Finance

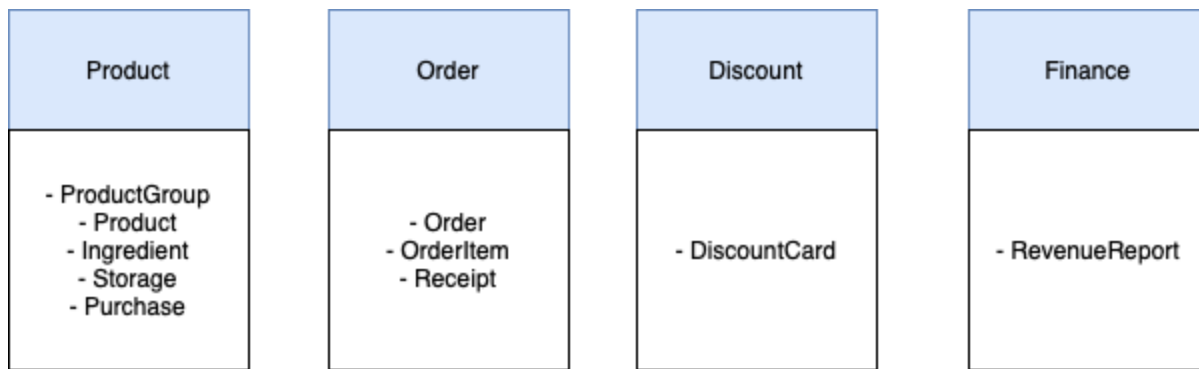
So we're grouping the classes based on their functionality. In our application, we have responsibilities, such as:

- Product management
 - Keeps track of our products and ingredients
- Order management
 - Stores the customers' orders
- Discount management
 - Handles discount cards and so on
- Finance
 - Creates reports about the business

Now let's put each class into one group:

- Product management
 - ProductGroup
 - Product
 - Ingredient
 - Storage
 - Purchase
- Order management
 - Order
 - OrderItem
 - Receipt
- Discount management
 - DiscountCard
- Finance
 - RevenueReport
 - CostOfRevenueReport

So, we basically namespaced our classes right? Yes, and in a classic monolith application, you create one repository and put all the classes in it. But in the microservices world, we create **a separate project for each namespace**.



What's exactly a service?

A service is a running Laravel instance, an application on its own, which usually has an API, and can communicate with the outside world. You create these groups, these boundaries around your functionality, and put each group into a different Laravel application with its own API.

It's not a composer package. It's an application. You can start it, stop it, deploy it at any time, call its API, run an artisan command in it, and so on.

What makes it micro?

In software development, size does matter. Usually the smaller, the better. The smaller your function, your class, the easier it is to maintain it. The same concept applies to services.

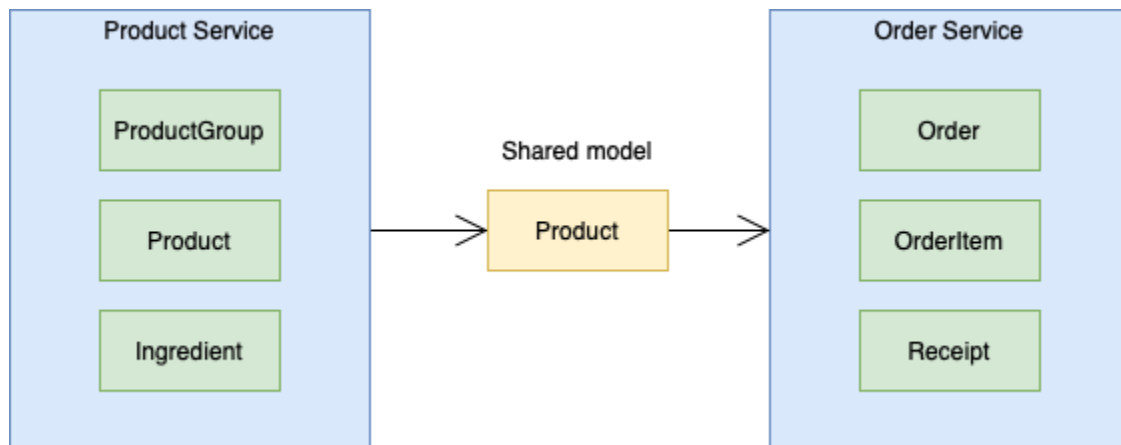
How small a service is? It's hard to tell exactly, but there's a good rule of thumb: it has to be small enough to completely rewrite it in one or two sprints. So if you have a team of 4 and you work in 2 weeks sprints, then you can rewrite the whole service in 2-4 weeks. By rewrite I mean, you delete the git repo and start from scratch. It really depends on the project, on the team, maybe it's 5000 lines of code for you, maybe it's 2000 for another team. By the way, the rewrite rule comes from Sam Newman's Building Microservices.

Alright, so we took our classes, put them into groups, and created 4 different services. How do they communicate with each other? How does the Finance service get the orders and products to calculate revenue? In the next few chapters we will talk about all of our options, but first, we need to talk about these groups a little bit.

Finding bounded context

On the previous pages, we talked about groups and grouping classes. But in the real world, we call these groups bounded contexts. The expression comes from Eric Evans' book called Domain-Driven Design. Every product has its own domain, and language, like the one in the restaurant example. Every domain has models in it, but you can always find a set of models that can be grouped together and you can draw a boundary around them. This boundary means that these models can live together without any other external dependency.

In every bounded context, there are some internal implementation details, and some internal models which you don't want to expose to the outside world. But it also has some shared models, which can be shared with the outside world. This is a very important concept. What it means is that you don't want to expose all of your Laravel Models from the Product service for example. You want to keep these models inside the product service because it has a lot of implementation details, and you want to create some leaner, transformed version for external usage. We will talk about it in more detail later in this book. For now, you can imagine something like this:



Where the green `Product` box is the internal model, a classic Laravel Model, and the yellow one is an external model, which can be shared with the order service, for example. In code, we'll use `DataTransferObjects` to represent these shared models.

Finding bounded context is the single most important thing when we talk about microservices. And unfortunately, it is the hardest skill to have. For example in the restaurant example, we have a model where:

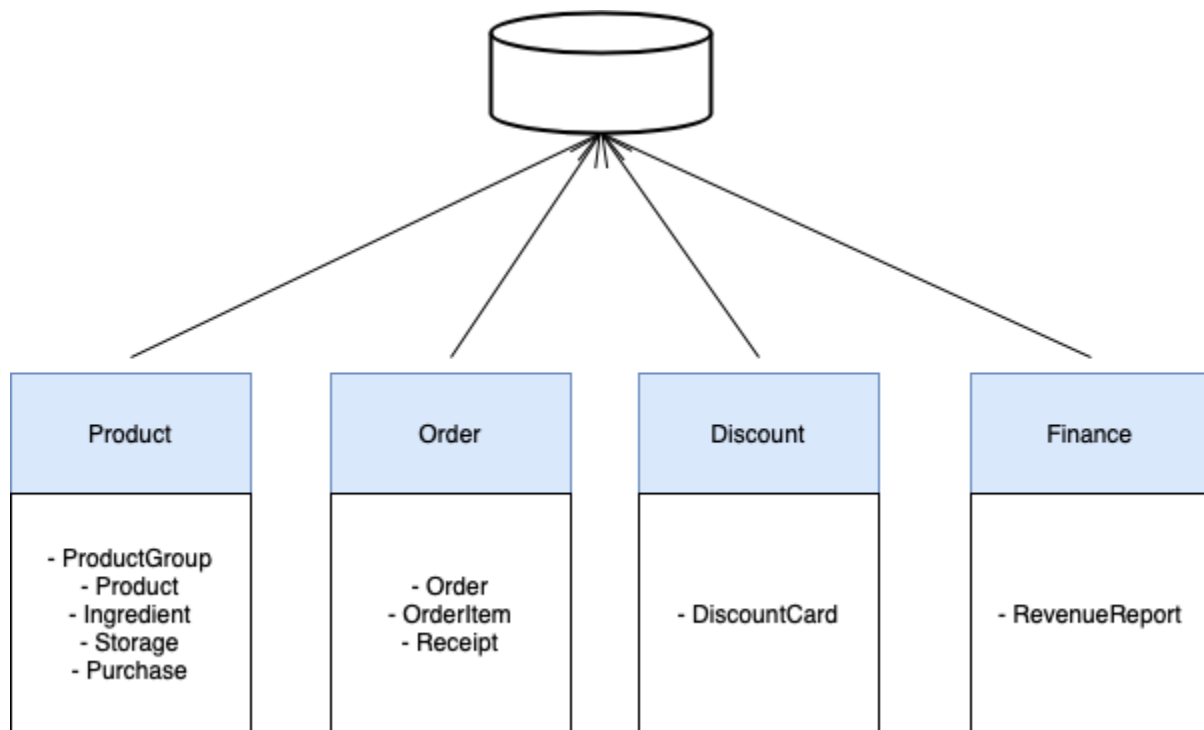
- `ProductGroup` contains products
- A `Product` is made of ingredients

- An Ingredient is stored in a Storage

And we have these models in the same service, in the Product service. But why don't we have a separate service for the whole storage management? And the answer is because this is just an example. Here's the thing: products and ingredients are closely related things, so it's logical to put them in one bucket. But in my experience warehouse management is complicated enough to have its own service. But maybe in this application, it's a simple thing so we don't want to complicate things. So it really depends on the product, the domain, and the functionality of our software.

Storing data in different services

The first big challenge that comes with microservices is storing data. This is harder than you think. The single most important thing is that you don't want to have only one database. So we won't do this:

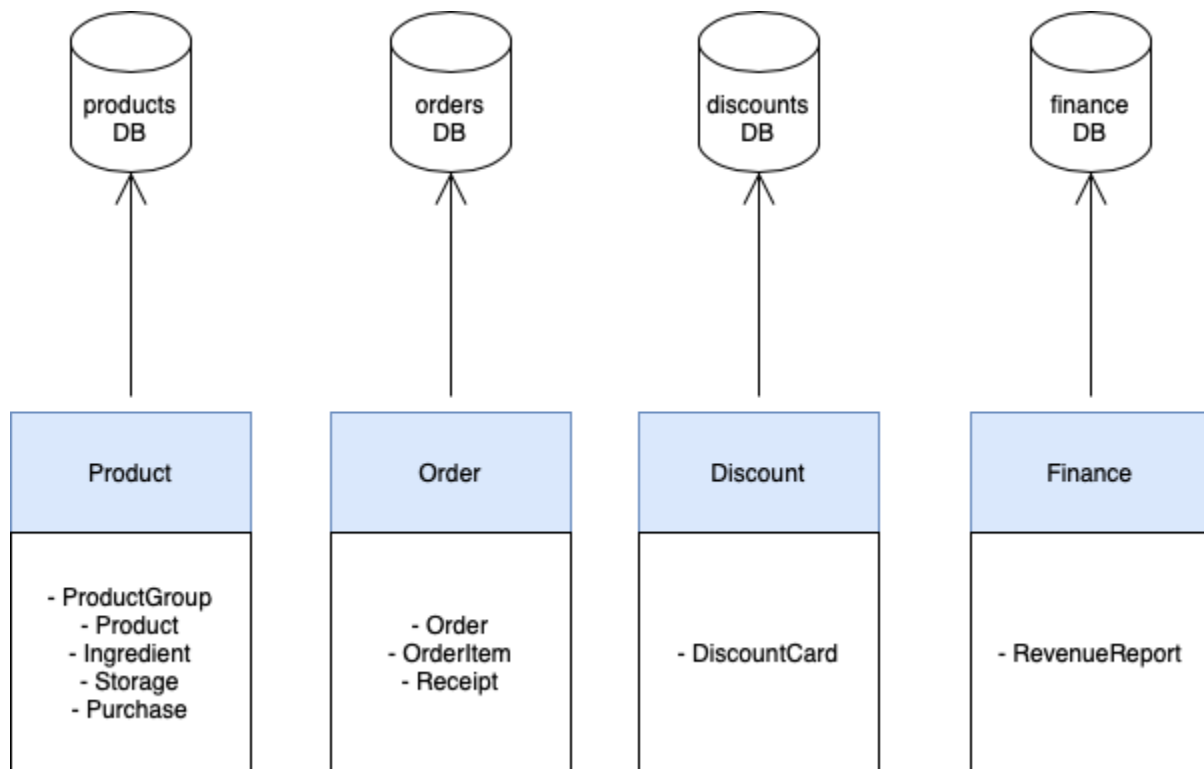


There are several problems here:

- **Tight coupling.** In this example, the Finance service needs to know every column and every detail about how a Product is stored in the DB. Finance is just a user of the product service, so it does not need to know if the ID is a string or an integer and so on.

- **Low cohesion.** Finance service will possibly break every time when someone changes the scheme of the products table. You make a change in the Product services and suddenly Finance stops working. This is bad.
- **Shared responsibility is bad.** Who's in charge of modifying the scheme? You can say it's easy, the scheme of products table is in the Product service, the scheme of orders table is in the Order service, and so on. It's easy to keep track when you have 4 services. But what if you have 132 services? In one database basically, there are no boundaries, no rules about how the data is related to services, and who is responsible to manage this data.
- **Too much dependency.** We want each service to run independently from other services. If we have a giant, shared database it's hard to achieve.
- And a bonus point: maybe we want a different type of database for a service, because it works better with NoSQL for example, meanwhile the rest of our services work better with SQL.

So here's what we want to do instead:



So every service has its own database (if it needs one). There are several benefits:

- **Loose coupling.** Much less coupling between services. Now Finance service does not depend on the scheme of products table. It only uses the shared models of the Product service.

- **High cohesion.** Now Finance service has no idea of the products table or products scheme. You can do whatever you want in the Product service and products DB, it will not break Finance.
- **Well-defined responsibilities.** Now the Product service is in charge of the products DB and no one else can touch it. Similarly, the Discount service is responsible for the discounts database and so on.
- **No dependencies.** Now every service runs independently from each other's data. They are just using the shared models and communicating with well-defined contracts (APIs and events).
- Now Finance can use MongoDB, while Product has a MySQL db, but Order has PostgreSQL. There are no technical limitations.

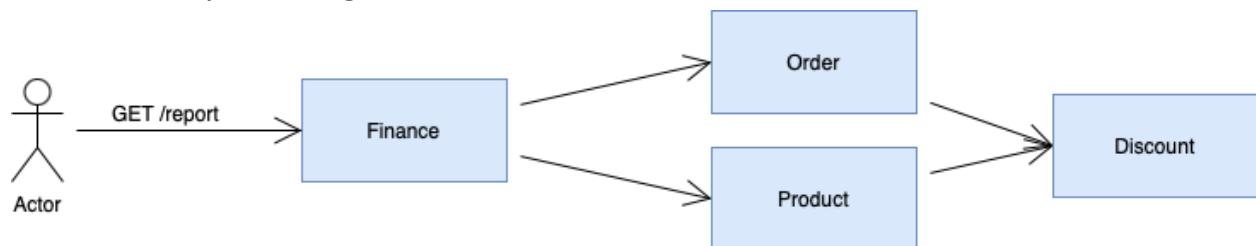
Communication between services

The other big challenge we have to solve is communication between services. So we split our application into different, small services. But we are no longer able to call functions between two services like we were in a monolith. In other words, how does the Finance service get the orders for a report? There are two ways to solve that problem (in fact there are more than two solutions, but we will focus on these two).

Sync communication

This is the easiest to understand and implement so we'll start here. We are using a request-response model to communicate between services. We are not talking about HTTP exclusively, but this is the most common one (other solutions can be RPC and protocol buffers).

Let me show you a diagram first:



Each arrow represents an HTTP API request. So whenever a service needs some data from another service it sends a request and gets back a response with the data. It's called sync communication because everything happens in order:

- The browser sends an HTTP request to the Finance service and waits for a response
- The Finance service sends a request to the Order service and waits for a response
- The Finance service sends a request to the Product service and waits for a response
- Both Order and Product service send a request to the Discount service

After every request is served, the Finance service does its calculations and responds to the user's request.

It has some advantages:

- Easy to understand. It's very similar to function calls in a monolith, except they are HTTP requests. But the flow of information is kind of the same.

- You can implement some services without a database. In the restaurant domain, the Finance service can be a good example. It does not necessarily require a database, because all the data that it needs is already there in some other services. So it's a possible solution to just request this data.

Besides, it is easy to understand and implement, but it has some major drawbacks:

- **Dependency** between services. This is an important one. We want to use microservices because we can develop services independently from each other, and we can deploy them whenever we want. If we couple them with HTTP calls it just makes life harder.
- **More sources of errors.** We make a lot of HTTP requests inside our services. What happens if the Order -> Discount request fails? Then the whole request chain fails. What happens if a service is down, and a timeout occurs? Then the whole request chain fails. This is gonna take us to our next drawback.
- **Latency.** We make **lots of requests**. Imagine we have just 30 services. We can easily make a request chain of 20-30 HTTP calls. It's just slow, way slower, than 20-30 function calls in a monolith.
- **Circular dependencies.** What happens if the Order service calls the Product service, then the Product service calls the Discount service, but for some reason, the Discount service also calls the Product service?



If you have a lot of services, and multiple teams working on them, it is very easy to do something like this. And now you have a circular dependency, and an out-of-memory or max execution time exceeded error message. These HTTP calls are very similar to function calls in a big monolith, where every module calls every other module.

Disclosure: Don't get me wrong, sync communication via HTTP is not evil by nature! You'll see later in this book, that we use this technique because it's simple and easy to use. In fact, sometimes you don't have another choice. Sometimes you just have to make an HTTP call, because it's synchronous, and you need an immediate response. But in my opinion, it's not a good solution, to build an entire system around it.

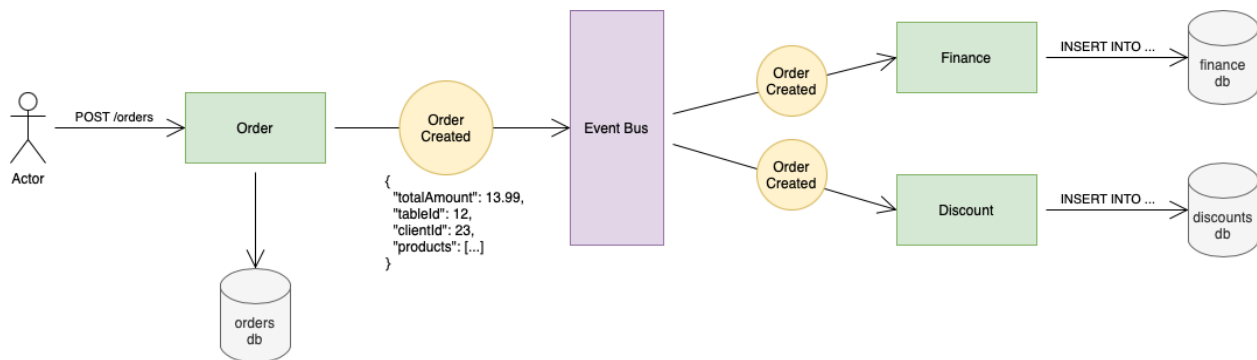
So what other alternative do we have?

Event-driven communication

In this communication model, we have to introduce a new component, called the event bus. We will talk about it in more detail later, and we will implement it, but for now, think about it as a broker who delivers messages between services.

In this model, we also rely on the concept that each service has its own database.

Let me show you an example:

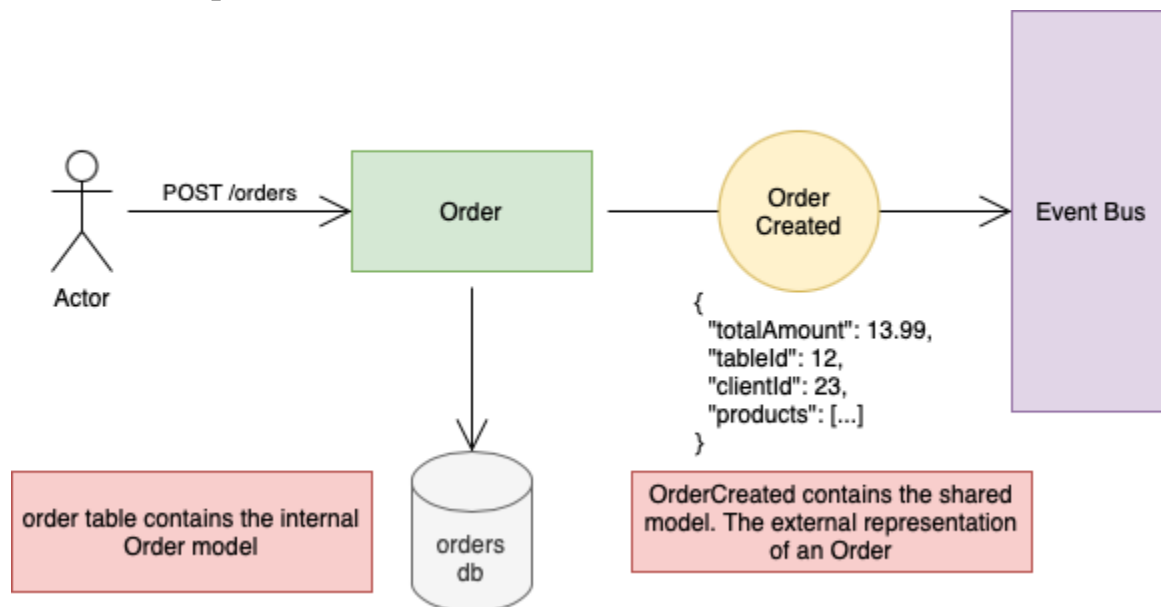


Let's review what is happening here:

- The user creates a new order
- The Order service receives an HTTP POST request with the order data
- It does its job, calculates some stuff, and writes it into the DB. So now the new order is created
- The Order service then **publishes** an event called *OrderCreated* to the Event Bus. It contains all of the important information about the order, such as the total amount, the table number, the customer, the product, etc.
- The Event Bus pushes the *OrderCreated* event to all the services that are interested in it. For example, the Finance service needs the newly created order to calculate the daily revenue. But maybe the Product service does not care about orders, so there is no need to push. (By the way, technically it can be a pull model also, where services pull the new events out of the event bus)
- The finance service gets the *OrderCreated* event, processes it, and makes an *insert* or an *update* on its own database.

So this is how every service builds its own database. By communicating through an event bus, and consuming interesting events. This way, every service has every important data it needs. No need to make an HTTP request from the Finance service to the Order service anymore. As the events come into the Event Bus, every service processes it and saves the important data.

One more important note:



When the Order API processes the POST request and saves an Order instance into the orders table, this is what we call the **internal Order model**. It's only visible to the Order service, and it may contain 20 columns.

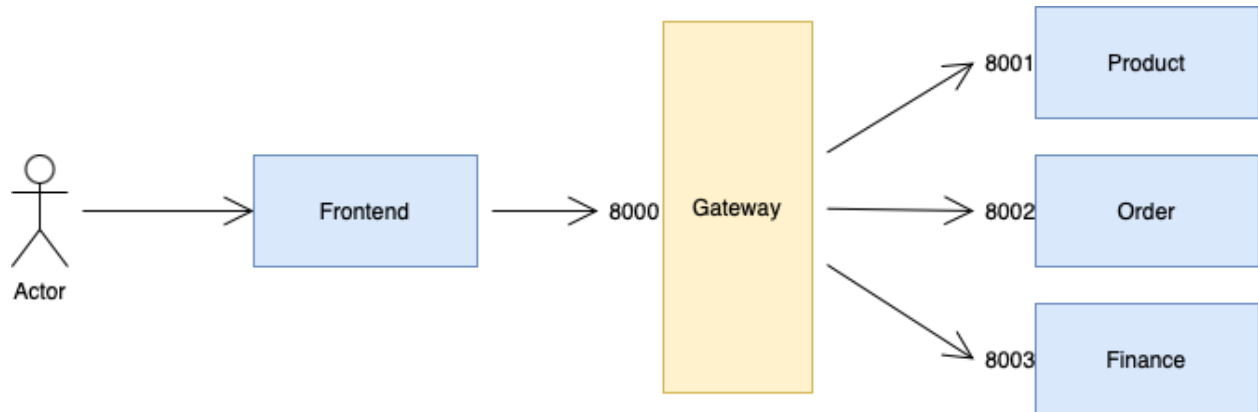
When the Order service publishes the *OrderCreated* event, it contains **the shared Order model**. It's visible to the whole system and may contain only 5-10 columns. Events and the Event Bus always contain language-independent formats, like JSON, so they can be processed by any language.

The important takeaway here is that only the shared, external model (DTO) can leave the service boundaries.

Frontend

So now we have a bunch of services running on a host on different ports. What about the frontend? How the frontend will know that when a user clicks on the "Create Order" button we need to send a request to localhost:8001/api/v1/posts, but when a user clicks on the "Create Product" button we need to use localhost:8002 because this is where the Product service listens?

The answer is that our frontend does not know anything about these different services and ports. We need a gateway between the FE and the services. Something like this:



So the gateway sits between the FE and the BE services. It acts as a reverse proxy. It takes requests from the FE, maps them to the right service, and sends the request to it.

You can implement an API gateway in (at least) two ways:

- Reverse proxy. You can use Nginx or haproxy. It's very simple and takes about 5 lines of config by service. It only proxies your requests.
- You can write your own gateway "service." It acts exactly like an Nginx reverse proxy, but it actually contains some code, so you can do extra tasks like:
 - Authenticate every request via an Auth service. And by the way, in the microservices world, you want to use stateless, jwt token-based authentication.
 - Serving the responses from the cache. If you have a gateway service, you can cache in one place, and you don't have to implement caching in every service. I'm talking about Redis or Memcached.

Each solution can work, but if you choose the second one, please don't use Laravel. Don't get me wrong, it's an excellent framework, but it's a behemoth for this job. It will be really slow compared to a Nodejs express or a Python flask gateway.

The promise of microservices

I hope all of this sounds good and architect-y, but there is one more important question. Why are we doing this?

So, here are some of the advantages of microservices:

- There is no huge codebase anymore. All of our services will be small, easy to reason about, and simple. This is a huge gain if you think about it.
- It's more resilient and reliable. Almost every error and bug has a scope of one service. So if something goes wrong in the Finance service, everything else will work properly. This is also a huge advantage. Basically, it's way harder to make a mistake that brings the whole application down.
- You can deploy small changes frequently. Because we have small services, it's very easy to deploy and ship new features. If you identify the right service boundaries, you can make new features and changes that only affect one or two services.
- Availability. You can use Docker Swarm or Kubernetes to orchestrate your services. This means that you can run mission-critical services in more instances, you can easily isolate worker processes, and you can auto-scale more effectively.
- You can use 4-5 member teams to develop and maintain two or three services. It's much easier to scale your organization.
- It's also easier to create autonomous and cross-functional teams. This means that every team can handle the whole lifecycle of a feature or a service, which includes:
 - Idea
 - Design
 - Development
 - QA
 - Deployment
 - Shipping to production
 - Maintenance
- The above point is possible because they only need to know a couple of services from bottom to top. It contains a few thousand lines of code. Not millions.
- You can use multiple languages, multiple frameworks, or technologies in your services. You can use Laravel in the Product service, but use Symfony in the Finance service. They are communicating via API calls and events, not function calls, so it doesn't matter what's inside a service. You can also write some Nodejs or Python services if you want. You can use MongoDB for a specific service. And so on.

- Fast and small pipelines. Each service is about a few hundred or thousand lines of code, so tests will be run fast, images will be built fast, and so on. There are no more 20, 30 minutes long pipelines.

But of course, microservices are no silver bullet, so there are also some drawbacks:

- Harder to understand. There are a lot of services communicating with each other. There are a lot of containers running, and multiple databases to migrate and maintain.
- Harder to debug. Let's say a user clicks a button, and the operation goes through 5 services. Now if something bad happens, you may have to debug 5 different Laravel projects. If you use event-driven communication it gets even worse, because it's asynchronous, so it's even harder to debug.
- Harder to monitor. You could have 30 services, and 75 running containers to monitor. You have to learn specific, and sometimes complicated tools to monitor your running application (like Prometheus).
- Harder to collect logs. Again, you have 75 running containers which dump logs to stdout. It gets even worse if you have more than one server (and you probably have more). Once again, you need to learn specific tools to see your logs (like ELK or EFK stack).
- You could run into problems that only come with distributed systems. For example, a service publishes a couple of events, and somehow it's processed in the wrong order, and now you have some data inconsistency in your databases. You have to debug multiple services, events, databases, and so on.
- Sometimes you have to make a change in every service. Let's say you have 14 services, and you found a great Laravel package, maybe some static code analysis package. You want to use it. But you want to use it in every service. So you have to install it in all 14 services. If you need this to run in your pipeline, you have to make changes to 14 pipelines.

As you can see there are some serious issues that come with microservices. Problems that you never have in a monolithic application. And you need to learn some new stuff to effectively operate this architecture.

But one thing is for sure: if you do it right, and you do it with the right application, it is worth it.

This is a sample chapter from the original book.
Get your copy here: <https://microservices-laravel.io>